

1 Exemplo de negociação em MASPY

Seguem um exemplo de utilização do framework MASPY com negociação.

O exemplo é ilustrado no Código 2. Ele apresenta a negociação entre vendedores (*sellers*) de passagens para viagens (*trip*) e compradores (*buyers*) que querem essas viagens. Os vendedores usam do ambiente website para anunciar suas viagens disponíveis, apresentando o número de paradas até o destino e o custo. Cada comprador possui suas preferências sobre esses valores e buscam a melhor viagem anunciada no website. A fig. 1 mostra a modelagem deste sistema.

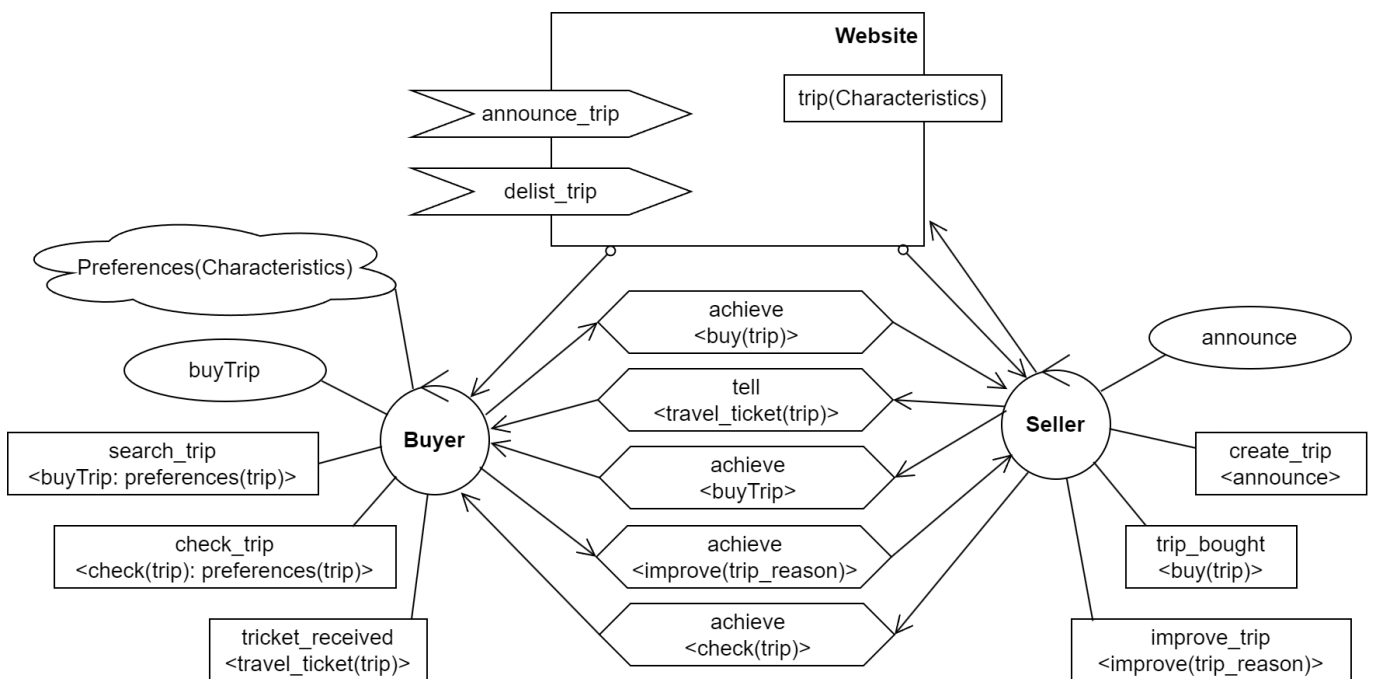


Figura 1: Modelagem do Exemplo de Negociação (MASPY)

O processo de negociação se desenrola da seguinte forma:

Sellers anunciam, no *website*, *trips*(viagens) com valor aleatório de paradas e custo. *Buyers* buscam neste *website* pela melhor oferta de *trip* disponível e verificam se ela é aceitável em relação a suas preferências.

- Se aceitam, eles informam o *seller* que deseja comprar a viagem.
- Se recusam, informam o *seller* como a oferta podia ser melhorada.

Se *Buyer* deseja comprar a viagem, *Seller* envia um *ticket* ao *buyer* caso a viagem foi comprada e ainda está disponível, e a remove do *website* ou informa que a que a viagem não existe mais, caso já tenha sido vendida.

Se *Buyer* pede por uma oferta melhor, *Seller* cria um novo anúncio de viagem quando pedido para melhorar, enviado este novo ao *buyer* que requisitou. Este então verifica novamente se a oferta é aceitável, se recusado novamente ele desiste, se aceito o processo indicado anteriormente é executado.

As Figs. 2 e 3 apresentam resultados da execução do sistema.

```

1 from maspy import *
2 from random import randint
3
4 # Definindo o ambiente website
5 class Website(Environment):
6     # Action de anunciar uma nova viagem
7     # trip e uma tuple com (stops, price)
8     def announce_trip(self, agt, trip):
9         self.print(f"Trip announced by {agt}: {trip}")
10        self.create(Percept("trip", {"seller": agt, "attributes": trip}))
11
12    # Action de remover anuncio de uma viagem
13    # trip e um dicionario {"seller": seller, "attributes": (stops, price)}
14    def delist_trip(self, agt, trip):
15        self.print(f"Trip delisted by {agt}: {trip['attributes']}")
16        self.delete(Percept("trip", trip))
17
18 #Definindo o agente vendedor
19 class Seller(Agent):
20     def __init__(self, agt_name=None):
21         super().__init__(agt_name)
22         self.add(Goal("announce")) # Goal Inicial
23
24     # Plano de criar viagens
25     # Gatilho: Ganhar objetivo "announce"
26     # Contexto: Nenhum
27     @pl(gain, Goal("announce"))
28     def create_trips(self, src):
29         for _ in range(5):
30             stops = randint(0,6)
31             price = randint(int(700/(stops+1)),1000)
32             trip = (stops, price)
33             self.print(f"Creating trip: {trip}")
34             self.announce_trip(trip) # executando a action de anunciar uma
viagem do website
35
36     # Plano de viagem comprada
37     # Gatilho: Ganhar objetivo "buy(Trip)"
38     # Contexto: Nenhum
39     @pl(gain, Goal("buy", "Trip"))
40     def trip_bought(self, src, trip):
41         # a var trip contem o argumento Trip do objetivo buy(Trip)
42         if self.has(Belief("trip", trip)): # verificando se o agente possui o
Belief trip(Trip)
43             self.print(f"Selling trip: {trip['attributes']} to {src}")
44             self.delist_trip(trip) # executando a action de remover anuncio de
uma viagem do website
45             self.send(src, tell, Belief("travel_ticket", trip)) # enviando o Belief
travel_ticket(Trip) a "src"
46         else:
47             self.print(f"Trip {trip['attributes']} not found for {src}")
48             self.send(src, achieve, Goal("buyTrip")) # enviando o objetivo buyTrip
a "src"
49
50     # Plano de melhorar uma viagem
51     # Gatilho: Ganhar objetivo "improve(Trip, Reason)"
52     # Contexto: Nenhum
53     @pl(gain, Goal("improve", ("Trip", "Reason")))
54     def improve_trip(self, src, improve_args):
55         # a var improve_args contem os argumentos Trip e Reason de objetivo

```

```

improve(Trip, Reason)
56     trip, reason = improve_args
57     match reason:
58         case "stops":
59             new_stop = trip['attributes'][0] - 1
60             new_price = trip['attributes'][1] + randint(100,200)
61         case "price":
62             new_stop = trip['attributes'][0] + 1
63             new_price = trip['attributes'][1] - randint(50,150)
64
65     new_trip = (new_stop, new_price)
66     self.print(f"Creating an improved trip: {trip['attributes']} -> {
new_trip} because of {reason} for {src}")
67     self.announce_trip(new_trip) # executando a action de anunciar uma
viagem do website
68     new_trip = {"seller": self.str_name, "attributes": new_trip}
69     self.send(src, achieve, Goal("check", new_trip)) # enviando o Goal check(
Trip) a "src"
70
71 # Definindo o agente comprador
72 class Buyer(Agent):
73     def __init__(self, agt_name, pref_stops, pref_price):
74         super().__init__(agt_name)
75         self.add(Belief("preferences", (pref_stops, pref_price))) # Belief Inicial
76         self.add(Goal("buyTrip")) # Goal Inicial
77
78     # Plano de procurar uma viagem
79     # Gatilho: Ganhar objetivo "buyTrip"
80     # Contexto: Preferences(Stops, Price)
81     @pl(gain, Goal("buyTrip"), Belief("preferences", ("Stops", "Price")))
82     def search_trip(self, src, prefs):
83         # prefs contem o argumento (Stops, Price) do objetivo preferences(Stops,
Price)
84         self.wait(1) # suspende o agente por 1 segundo, ciclo de raciocineo
continua
85         # Busca por todas as viagens percebidas
86         trips = self.get(Belief("trip", "Characteristics"), all=True, ck_src=False)
87
88         if trips is None:
89             self.print(f"No trips available")
90             self.wait(1)
91             self.add(Goal("buyTrip"))
92             return
93
94         best_score = 0
95         best_trip = Belief
96         best_reason = ""
97         for trip in trips:
98             assert isinstance(trip, Belief)
99             score, reason = self.evaluate_trip(trip.args, prefs)
100             if score > best_score:
101                 best_score = score
102                 best_trip = trip
103                 best_reason = reason
104
105         seller = best_trip.args['seller']
106         atts = best_trip.args['attributes']
107         if best_score > 2:
108             self.print(f"Trip from {seller} accepted: {atts}")
109             self.send(seller, achieve, Goal("buy", best_trip.args)) # enviando o

```

```

110         else:
111             self.print(f"Trip from {seller} rejected: {atts}, asking to improve
{best_reason}")
112             self.send(seller, achieve, Goal("improve", (best_trip.args, best_reason)
)) # enviando o Goal improve(Trip, Reason) ao vendedor
113
114         # Plano de checar uma viagem
115         # Gatilho: Ganhar objetivo check(Trip)
116         # Contexto: Preferences(Stops, Price)
117         @pl(gain, Goal("check", "Trip"), Belief("preferences", ("Stops", "Price")))
118         def check_trip(self, src, trip, prefs):
119             # a var trip contem o argumento Trip do objetivo check(Trip)
120             # prefs contem o argumento (Stops, Price) do objetivo preferences(Stops,
Price)
121             score, reason = self.evaluate_trip(trip, prefs)
122             seller = trip['seller']
123             atts = trip['attributes']
124             if score > 1.8:
125                 self.print(f"Trip from {seller} accepted: {atts}")
126                 self.send(src, achieve, Goal("buy", trip)) # enviando o Goal buy(Trip)
ao "src"
127             else:
128                 self.print(f"Trip from {seller} rejected: {atts} because of {reason
}. Giving up")
129                 self.stop_cycle() # parar ciclo de raciocineo do agente
130
131         # Nao eh um plano do agente, apenas uma funcao para avaliar uma viagem
132         def evaluate_trip(self, trip: dict, prefs: tuple) -> tuple[float, str]:
133             stops, price = trip['attributes']
134             pref_stops, pref_price = prefs
135
136             # *_diff ganham valores de 0 a 2
137             stop_diff = 1 - (stops - pref_stops) / 6
138             price_diff = 1 - (price - pref_price) / 1000
139             # score maximo igual a 4
140             score = stop_diff + price_diff
141
142             if stop_diff < price_diff:
143                 improve = "stops"
144             else:
145                 improve = "price"
146
147             return score, improve
148
149         # Plano de receber um bilhete de viagem
150         # Gatilho: Ganhar belief "travel_ticket(Trip)"
151         # Contexto: Nenhum
152         @pl(gain, Belief("travel_ticket", "Trip"))
153         def ticket_received(self, src, trip):
154             # a var trip contem o argumento Trip do belief travel_ticket(Trip)
155             self.print(f"Received travel ticket for trip {trip['attributes']} from {
src}")
156             self.stop_cycle() # parar ciclo de raciocineo do agente
157
158     if __name__ == "__main__":
159         ws = Website()
160         agent_list: list = [Seller() for _ in range(2)]
161
162         for _ in range(3):

```

```

163     pref_stops = randint(0,6)
164     pref_price = randint(int(700/(pref_stops+1)),1000)
165     buyer = Buyer("Buyer", pref_stops, pref_price)
166     agent_list.append(buyer)
167
168     Admin().connect_to(agent_list,ws)
169     Admin().report = True
170     Admin().start_system()

```

Código 1: Exemplo 1 (MASPY)

```

PS M:\Codes> Python ./ex-buying-trip.py
Starting MASPY Program
# Admin #> Starting Agents
Agent: Seller_1> Creating trip: (5, 994)
Environment: Website> Trip announced by Seller_1: (5, 994)
Agent: Seller_2> Creating trip: (3, 846)
Agent: Seller_1> Creating trip: (4, 435)
Environment: Website> Trip announced by Seller_2: (3, 846)
Environment: Website> Trip announced by Seller_1: (4, 435)
Agent: Seller_2> Creating trip: (5, 129)
Environment: Website> Trip announced by Seller_2: (5, 129)
Agent: Seller_1> Creating trip: (5, 899)
Environment: Website> Trip announced by Seller_1: (5, 899)
Agent: Seller_2> Creating trip: (5, 566)
Agent: Seller_1> Creating trip: (1, 552)
Environment: Website> Trip announced by Seller_2: (5, 566)
Environment: Website> Trip announced by Seller_1: (1, 552)
Agent: Seller_2> Creating trip: (1, 798)
Agent: Seller_1> Creating trip: (6, 522)
Environment: Website> Trip announced by Seller_1: (6, 522)
Environment: Website> Trip announced by Seller_2: (1, 798)
Agent: Seller_2> Creating trip: (1, 430)
Environment: Website> Trip announced by Seller_2: (1, 430)
Agent: Buyer_2> Trip from Seller_2 accepted: (1, 430)
Agent: Buyer_1> Trip from Seller_2 rejected: (1, 430), asking to improve price
Agent: Buyer_3> Trip from Seller_2 accepted: (1, 430)
Agent: Seller_2> Selling trip: (1, 430) to Buyer_2
Environment: Website> Trip delisted by Seller_2: (1, 430)
Agent: Seller_2> Creating an improved trip: (1, 430) -> (2, 292) because of price for Buyer_1
Agent: Buyer_2> Received travel ticket for trip (1, 430) from Seller_2
Environment: Website> Trip announced by Seller_2: (2, 292)
Agent: Seller_2> Trip (1, 430) not found for Buyer_3
Agent: Buyer_1> Trip from Seller_2 rejected: (2, 292) because of stops. Giving up
Agent: Buyer_3> Trip from Seller_2 accepted: (2, 292)
Agent: Seller_2> Selling trip: (2, 292) to Buyer_3
Environment: Website> Trip delisted by Seller_2: (2, 292)
Agent: Buyer_3> Received travel ticket for trip (2, 292) from Seller_2
# Admin #> [Closing System]
Still running agent(s):
Seller_1 | Seller_2 |
Ending MASPY Program
Making System Report...
Managing Data...
Writing to Excel...
System Report Completed
PS M:\Codes>

```

Figura 2: Execução 1 do Exemplo com negociação em MASPY

```

6 PS M:\Codes> Python ./ex-buying-trip.py
Starting MASPYPY Program
# Admin #> Starting Agents
Agent: Seller_1> Creating trip: (5, 639)
Environment: Website> Trip announced by Seller_1: (5, 639)
Agent: Seller_1> Creating trip: (5, 307)
Environment: Website> Trip announced by Seller_1: (5, 307)
Agent: Seller_2> Creating trip: (5, 169)
Agent: Seller_1> Creating trip: (0, 947)
Environment: Website> Trip announced by Seller_2: (5, 169)
Environment: Website> Trip announced by Seller_1: (0, 947)
Agent: Seller_2> Creating trip: (4, 211)
Agent: Seller_1> Creating trip: (6, 254)
Environment: Website> Trip announced by Seller_2: (4, 211)
Environment: Website> Trip announced by Seller_1: (6, 254)
Agent: Seller_2> Creating trip: (2, 469)
Agent: Seller_1> Creating trip: (1, 420)
Environment: Website> Trip announced by Seller_1: (1, 420)
Environment: Website> Trip announced by Seller_2: (2, 469)
Agent: Seller_2> Creating trip: (5, 277)
Environment: Website> Trip announced by Seller_2: (5, 277)
Agent: Seller_2> Creating trip: (3, 681)
Environment: Website> Trip announced by Seller_2: (3, 681)
Agent: Buyer_2> Trip from Seller_1 accepted: (1, 420)
Agent: Buyer_1> Trip from Seller_1 accepted: (1, 420)
Agent: Buyer_3> Trip from Seller_1 accepted: (1, 420)
Agent: Seller_1> Selling trip: (1, 420) to Buyer_2
Environment: Website> Trip delisted by Seller_1: (1, 420)
Agent: Buyer_2> Received travel ticket for trip (1, 420) from Seller_1
Agent: Seller_1> Trip (1, 420) not found for Buyer_1
Agent: Seller_1> Trip (1, 420) not found for Buyer_3
Agent: Buyer_1> Trip from Seller_2 rejected: (2, 469), asking to improve stops
Agent: Buyer_3> Trip from Seller_2 rejected: (2, 469), asking to improve price
Agent: Seller_2> Creating an improved trip: (2, 469) -> (1, 635) because of stops for Buyer_1
Environment: Website> Trip announced by Seller_2: (1, 635)
Agent: Seller_2> Creating an improved trip: (2, 469) -> (3, 387) because of price for Buyer_3
Environment: Website> Trip announced by Seller_2: (3, 387)
Agent: Buyer_1> Trip from Seller_2 accepted: (1, 635)
Agent: Seller_2> Selling trip: (1, 635) to Buyer_1
Environment: Website> Trip delisted by Seller_2: (1, 635)
Agent: Buyer_3> Trip from Seller_2 accepted: (3, 387)
Agent: Buyer_1> Received travel ticket for trip (1, 635) from Seller_2
Agent: Seller_2> Selling trip: (3, 387) to Buyer_3
Environment: Website> Trip delisted by Seller_2: (3, 387)
Agent: Buyer_3> Received travel ticket for trip (3, 387) from Seller_2
# Admin #> [Closing System]
Still running agent(s):
Seller_1 | Seller_2 |
Ending MASPYPY Program
Making System Report...
Managing Data...
Writing to Excel...
System Report Completed
PS M:\Codes>

```

Figura 3: Execução 2 do Exemplo com negociação em MASPYPY